

THỬ NGHIỆM KHẢ THI (PROOF OF CONCEPT - PoC)

Tính Khả thi Kiến trúc: Trình xem PDF An toàn & Anti-Leakage DRM cho LIBIF

Loại tài liệu: Kế hoạch Khả thi Thử nghiệm (PoC)

Dự án: LIBIF — Hệ thống Số hóa Thư viện & Quản lý Tài liệu Thông minh

Mục tiêu Cốt lõi: Xác minh tính khả thi kỹ thuật trong việc ngăn chặn tải xuống PDF thô và rò rỉ tài liệu không kiểm soát

Trạng thái: PHƯƠNG PHÁP ĐỀ XUẤT KHẢ THI

Ngày: 24 tháng 07, 2026

1. Mục tiêu

Mục tiêu của Thử nghiệm Khả thi (PoC) này là chứng minh cách **LIBIF** giải quyết thách thức bảo mật cốt lõi: **ngăn chặn việc tải xuống tài liệu PDF trái phép, sao chép văn bản, in ấn và rò rỉ tệp thô** trên trình duyệt web tiêu chuẩn mà không yêu cầu người dùng cài đặt các tiện ích mở rộng (plugin) của bên thứ ba.

2. Vấn đề Hài hám Nhất: Rò rỉ PDF Không Kiểm soát

2.1 Tại sao Quy trình Hiện tại Thất bại

Trong các ứng dụng thư viện thông thường hoặc các thiết lập tự phát (ví dụ: Google Drive, nhúng PDF thông thường, hoặc dùng iframe thô `<iframe src="book.pdf">`), tài liệu bị lộ dưới dạng các tệp PDF thô.

Người dùng có thể dễ dàng:

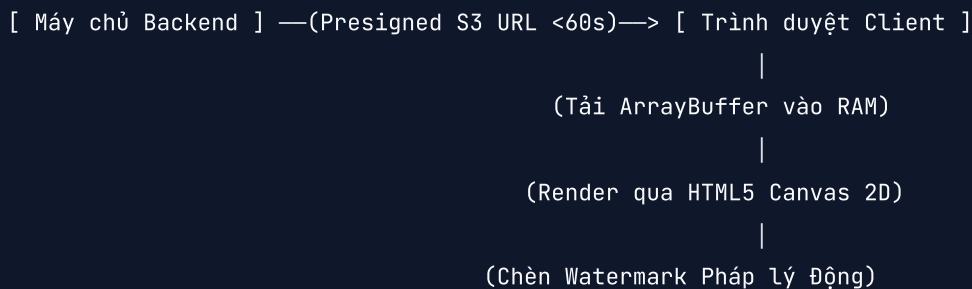
- Kiểm tra thẻ `Network` trong công cụ nhà phát triển của trình duyệt hoặc mã nguồn HTML để trích xuất đường dẫn `.pdf` trực tiếp.
- Tải xuống tệp PDF nguyên vẹn và chia sẻ tự do qua Zalo, Telegram hoặc email.
- Bôi đen và sao chép văn bản nhạy cảm trực tiếp từ DOM của trang web.

2.2 Thách thức Cốt lõi

Để đáp ứng yêu cầu bản quyền, LIBIF phải cho phép sinh viên đọc sách trên trình duyệt 24/7 nhưng đồng thời tuyệt đối ngăn chặn việc tải về hoặc trích xuất tệp `.pdf` thô.

3. Giải pháp & Kiến trúc Đề xuất

Để giải quyết thách thức này, LIBIF đề xuất cơ chế bảo mật 4 bước:



1. URL Presigned S3 Thời hạn Ngắn (< 60s TTL)

Thay vì dùng URL public bucket, backend tạo ra các URL S3 presigned tạm thời có hiệu lực **dưới 60 giây**. Ngay khi dữ liệu được tải vào bộ nhớ trình duyệt, liên kết sẽ hết hạn lập tức, ngăn chặn việc chia sẻ liên kết.

2. Tải ArrayBuffer Trực tiếp vào Bộ nhớ RAM

Client tải dữ liệu nhị phân của tài liệu trực tiếp vào bộ nhớ RAM trình duyệt (`ArrayBuffer`). Tệp PDF không bao giờ được lưu thành tệp trên đĩa cứng cục bộ, cũng như không bị lộ qua các URL `blob:` cố định.

3. Render Điểm ảnh HTML5 Canvas

Sử dụng `pdfjs-dist`, các trang sách được vẽ trực tiếp lên các thẻ HTML5 `<canvas>` dưới dạng các điểm ảnh 2D:

- **Không có Node Văn bản DOM:** Công cụ Inspect Element của trình duyệt không tìm thấy bất kỳ node HTML văn bản nào có thể đọc được.
- **Không thể Lưu Ảnh Trực tiếp:** Thẻ canvas ngăn chặn hành động "Save Image As" tiêu chuẩn khi vô hiệu hóa chuột phải.

4. Bắt Sự kiện DOM & Chống Sao chép

Các hàm lắng nghe sự kiện ở phía client bắt và chặn các hành vi của trình duyệt:

- Vô hiệu hóa menu chuột phải (`contextmenu`).
- Chặn các phím tắt tải xuống và in ấn (`Ctrl+S`, `Ctrl+P`, `Ctrl+U`, `F12`).
- Ẩn giao diện xem canvas khi người dùng cố tình in ấn thông qua quy tắc CSS `@media print`.

5. Watermark Pháp lý Động

Trong quá trình render canvas, một lớp phủ watermark trong suốt được vẽ đè lên trang sách bao gồm:

- Mã số Sinh viên (MSSV)
- Địa chỉ IP Client
- Dấu thời gian (Timestamp)

Nếu người dùng chụp ảnh màn hình bằng thiết bị vật lý bên ngoài, nguồn gốc rõ ràng sẽ lập tức bị truy vết.

4. Thiết kế Mã nguồn Khả thi

Dưới đây là cấu trúc cài đặt khái niệm cho component trình đọc được đề xuất:

```

import React, { useEffect, useRef } from 'react';
import * as pdfjsLib from 'pdfjs-dist';

interface SecureViewerProps {
  bookId: string;
  userMetadata: { studentId: string; ip: string };
  getTemporaryUrl: (id: string) => Promise<string>;
}

export const SecurePdfViewer: React.FC<SecureViewerProps> = ({ bookId, userMetadata, getT
  const canvasRef = useRef<HTMLCanvasElement | null>(null);

  useEffect(() => {
    const loadDocument = async () => {
      // 1. Get temporary URL (<60s)
      const tempUrl = await getTemporaryUrl(bookId);

      // 2. Fetch binary stream into RAM
      const res = await fetch(tempUrl);
      const buffer = await res.arrayBuffer();

      // 3. Render page onto Canvas
      const pdf = await pdfjsLib.getDocument({ data: buffer }).promise;
      const page = await pdf.getPage(1);

      const canvas = canvasRef.current;
      if (!canvas) return;
      const ctx = canvas.getContext('2d');
      if (!ctx) return;

      const viewport = page.getViewport({ scale: 1.5 });
      canvas.width = viewport.width;
      canvas.height = viewport.height;

      await page.render({ canvasContext: ctx, viewport }).promise;

      // 4. Draw Dynamic Forensic Watermark
      ctx.fillStyle = 'rgba(180, 0, 0, 0.15)';
      ctx.font = '16px sans-serif';
      ctx.fillText(`LICENSED TO: ${userMetadata.studentId} | IP: ${userMetadata.ip}`, 50,
    };

    loadDocument();
  }, [bookId]);

  // 5. Intercept Copy / Print / Context Menu
  useEffect(() => {
    const disableActions = (e: KeyboardEvent) => {
      if ((e.ctrlKey || e.metaKey) && ['s', 'p', 'u', 'c'].includes(e.key.toLowerCase()))
        e.preventDefault();
    }
  });

```

```
    window.addEventListener('keydown', disableActions);
    return () => window.removeEventListener('keydown', disableActions);
  }, []);

  return (
    <div style={{ userSelect: 'none' }}>
      <canvas ref={canvasRef} />
    </div>
  );
};
```

5. Tiêu chí Xác minh Tính Khả thi

Để xác minh phương pháp này trong quá trình phát triển (Sprint 3), việc cài đặt sẽ được đánh giá dựa trên các tiêu chí xác minh sau:

Mục tiêu Kiểm thử	Kết quả Mục tiêu
Trích xuất URL	Các URL presigned được trích xuất phải hết hạn trong vòng 60 giây và thất bại khi tái sử dụng.
Tải xuống Trực tiếp	Không tồn tại nút tải tệp .pdf trực tiếp hay liên kết thô trong mã nguồn DOM.
Sao chép Văn bản	Menu chuột phải và thao tác bôi đen sao chép Ctrl+C hoàn toàn bị vô hiệu hóa.
In ấn trên Trình duyệt	Ctrl+P chỉ in ra trang trắng nhờ vào quy tắc CSS @media print .
Khả năng Truy vết	Ảnh chụp màn hình hoặc ảnh chụp thực tế hiển thị rõ ràng watermark MSSV và IP.

6. Kết luận

Bằng cách kết hợp **URL S3 presigned thời hạn ngắn**, **tải ArrayBuffer trực tiếp vào RAM**, **render điểm ảnh trên HTML5 Canvas**, và **chèn watermark động**, LIBIF đã đưa ra một giải pháp rõ ràng, vững chắc về mặt lý thuyết và hoàn toàn khả thi để ngăn ngừa rò rỉ PDF mà không cần đến các plugin phức tạp trên máy tính.